

COSC 612/ AIT 624

Software Process

Dr. Akshita Maradapu Vera Venkata Sai
Department of Computer and Information Sciences
Towson University

Outline

- Software process models
- Process activities
- Coping with change
- Process improvement

Software Process

What is Software Process?

- “A software process is a set of **related activities** that leads to the **production** of a **software product**.”



These activities may involve :

- the development of the software from the scratch (or)
- modifying an existing system.

Software Process

- Any software process must include the following four activities:
 - Software specification (or requirements engineering)
 - Software design and implementation
 - Software verification and validation (testing)
 - Software evolution (software maintenance)

Software Process Activities

Software Processes Fundamentals Activities

- **Software Specification:**
 - The functionality of the software and constraints on its operation must be defined
- **Software Design and Implementation:**
 - The software is to be designed and programmed



Software Process Activities

Software Processes Fundamentals Activities

- **Software verification and validation**
The software must conform to its specification and meet the customer needs.
- **Software evolution (software maintenance):**
The software must conform to its specification and meet the customer needs.



Software Process Descriptions

- When we talk about a process, we usually talk about the activities in it.
However,

a process also includes the process description

Software Process Description

- **Products:** The outcomes of an activity.
 - For example, the outcome of architectural design maybe a model for the software architecture.
- **Roles:** The responsibilities of the people involved in the process.
 - For example, the project manager, programmer, etc.
- **Pre and post conditions:** The conditions: must be true before and after an activity.
 - For example:
 - the pre-condition of the architectural design is the requirements have been approved by the customer,
 - While the post condition is the diagrams describing the architectural have been reviewed.

Plan-Driven and Agile Processes

Plan-driven processes

- the process activities are planned in advance and progress is measured against this plan.

In agile processes

- planning is incremental and it is easier to change the process to reflect changing customer requirements.

In practice

- most practical processes include elements of both plan-driven and agile approaches.

There are no right or wrong software processes.

Software Process Models

Software Development Life Cycle (SDLC)

- Simplified Representation of a software process
- Each model shows a process in a different perspective – therefore, has partial information.
- Different models show different abstractions of the process
- Generic models :
 - Waterfall model – First software model
 - Incremental model – Most common model
 - Integration and Configuration
 - RUP (Rational Unified Process) – Combination of all the above models

Software Development Life Cycle (SDLC)

- RUP – Rational Unified Process
- This includes elements of all general process models
- It has three perspectives :
 - Dynamic – shows the phases of the model in time
 - Static – shows each process activity
 - Practice – good practices to be used in the process

Waterfall model

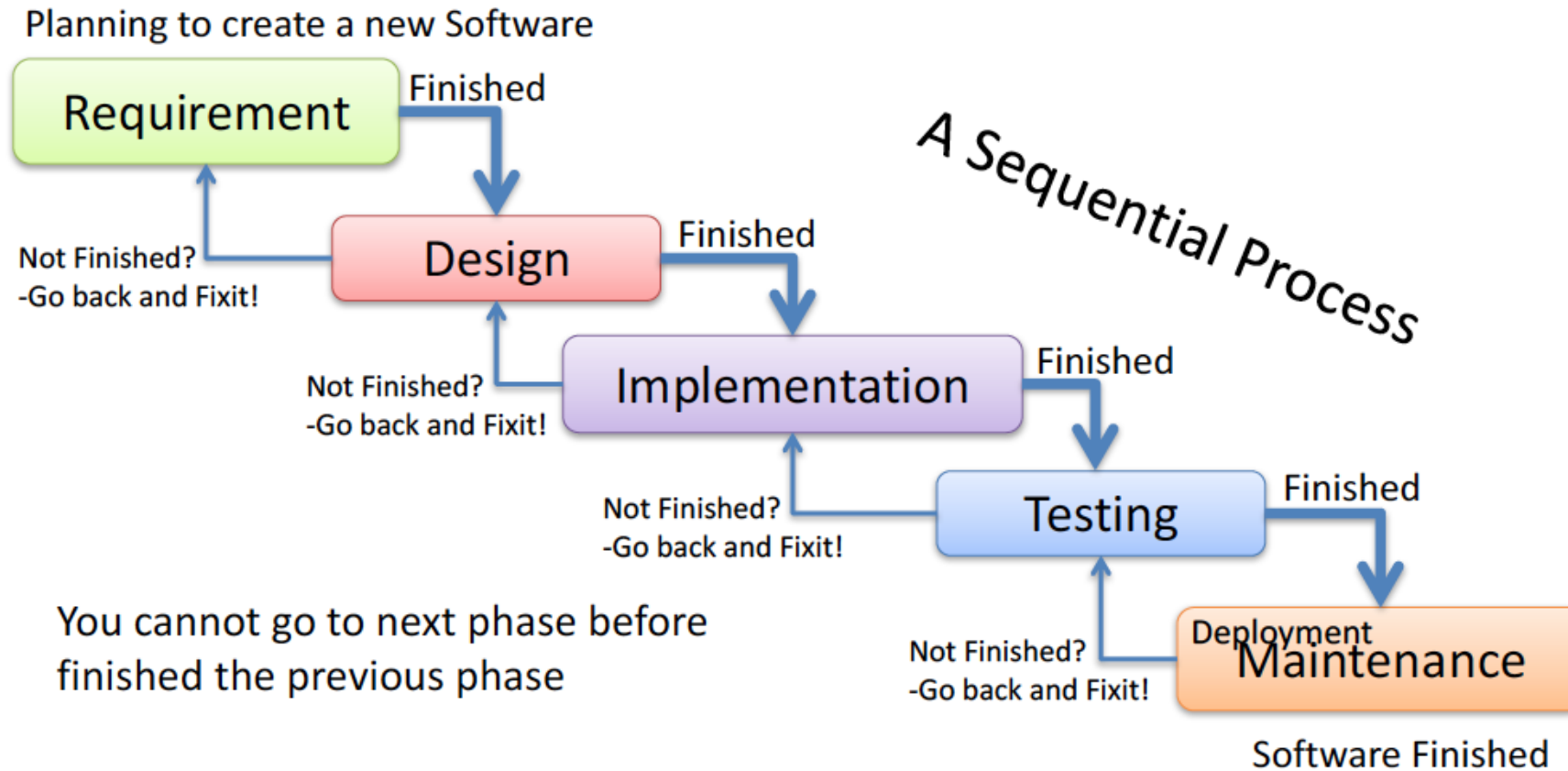
Waterfall Model

Water Fall Model

- One of the first life-cycle models (Royce, 1970)
- The waterfall development model originates in the manufacturing and construction industries
- Very common, very criticized



Waterfall Model



Waterfall Model

Features of Water Fall Model

A Water Fall Model is easy to flow.



It can be implemented for any size of project.



Every stage has to be done separately at the right time so you cannot jump stages

Documentation is produced at every stage of a waterfall model allowing people to understand what has been done.



Testing is done at every stage..

Waterfall Model

Why is the waterfall model
so criticized?

Waterfall Model

- The main drawback

- ✓ – Software **requirements change**, hard to sign-off on a SRS.
- ✓ – **Early commitment**. Changes at the end, large impact.
- ✓ – **Feedback** is needed to understand a phase. E.g. implementation is needed to understand some design.
- Difficult to **estimate time** and cost for the phases.
- Handling **risks** are not part of the model. Pushes the risks forward.
- Software **"is not"** developed in such a way. It evolves when problems are more understood.

Waterfall Model

- The main Pros

- ✓ + Simple, manageable and easy to understand
- ✓ + Fits to common project management practices (milestones, deliverables etc.)
- ✓ + Focus on requirements and design at beginning, save money and time at the end
- + Can be suitable for short projects (some weeks)
- ✓ + Can be suitable for "stable" projects, where requirements do not change
- + Focus on documents, saves knowledge which can be reused by other people.
- + Widely used, e.g. US Department of Defense
- ✓ + Can be suitable for fixed-price contracts

Waterfall Model

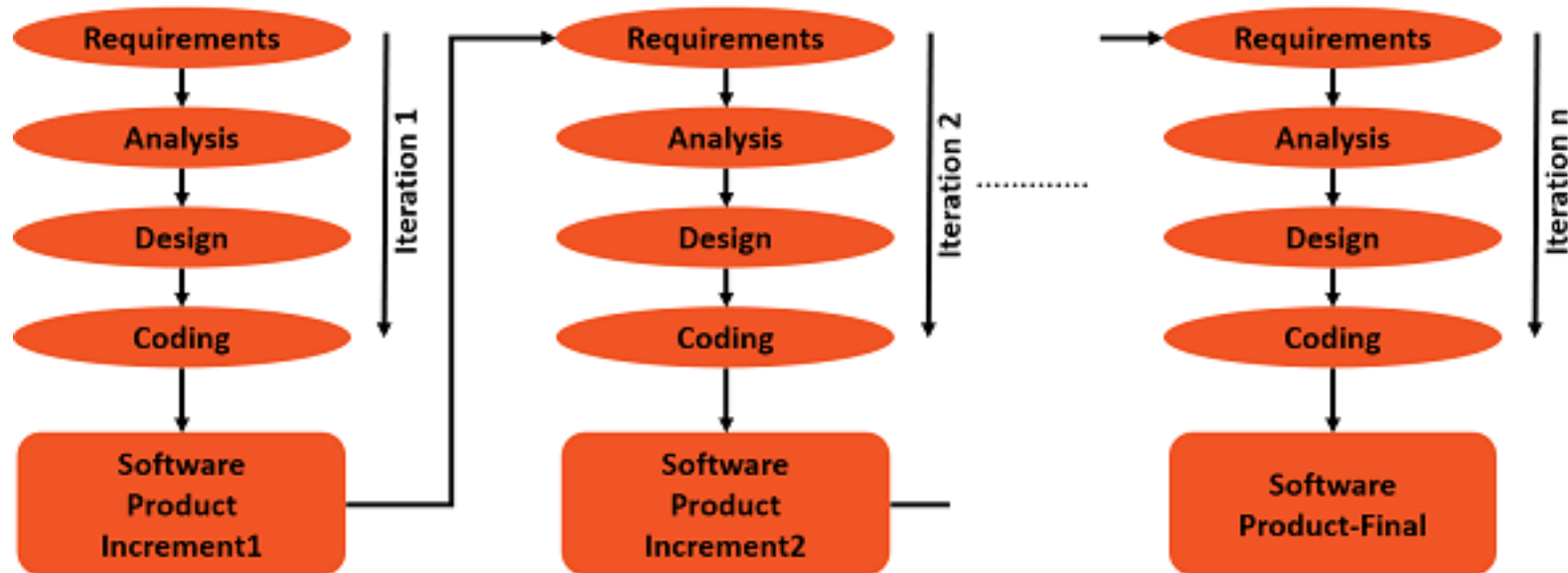
- When to use?
 - When the requirements are well-understood
 - When changes will be fairly limited during the design process.
 - Few business systems have stable requirements.
 - When large systems engineering projects are developed at several sites.
 - the waterfall model helps coordinate the work.

Mostly used in the development of software systems that have stringent **safety**, **reliability**, or **security** requirements

Incremental Development

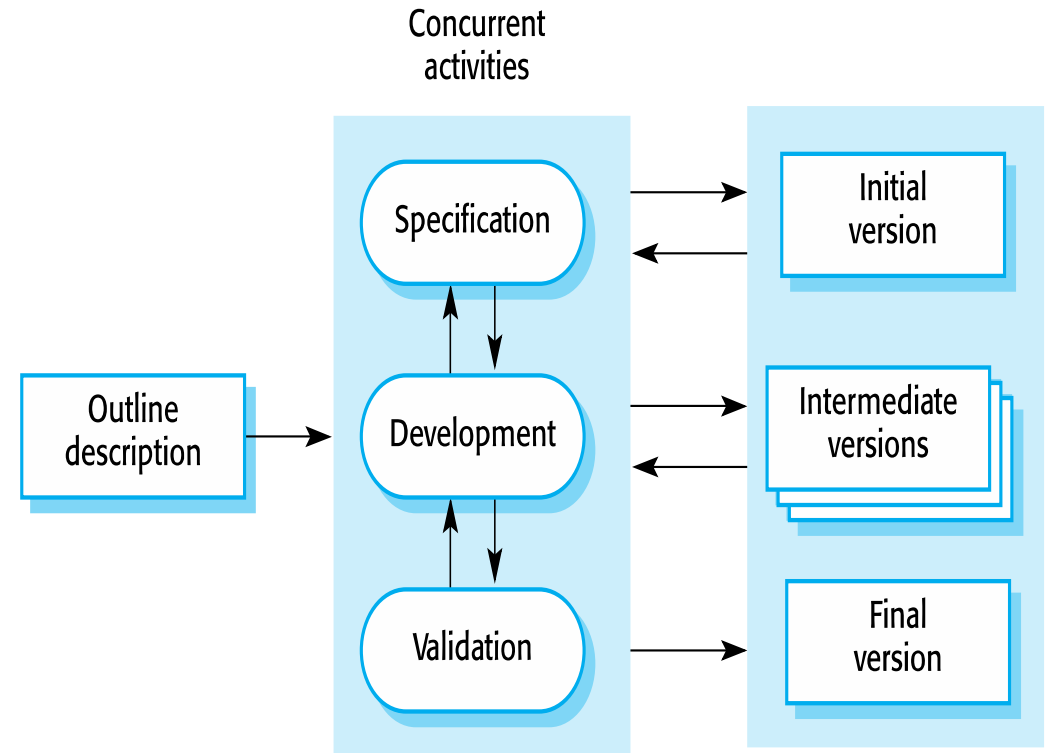
Incremental Development

- Incremental development partitions a system by functionality
- Early release starts with small, functional subsystem, later releases add functionality



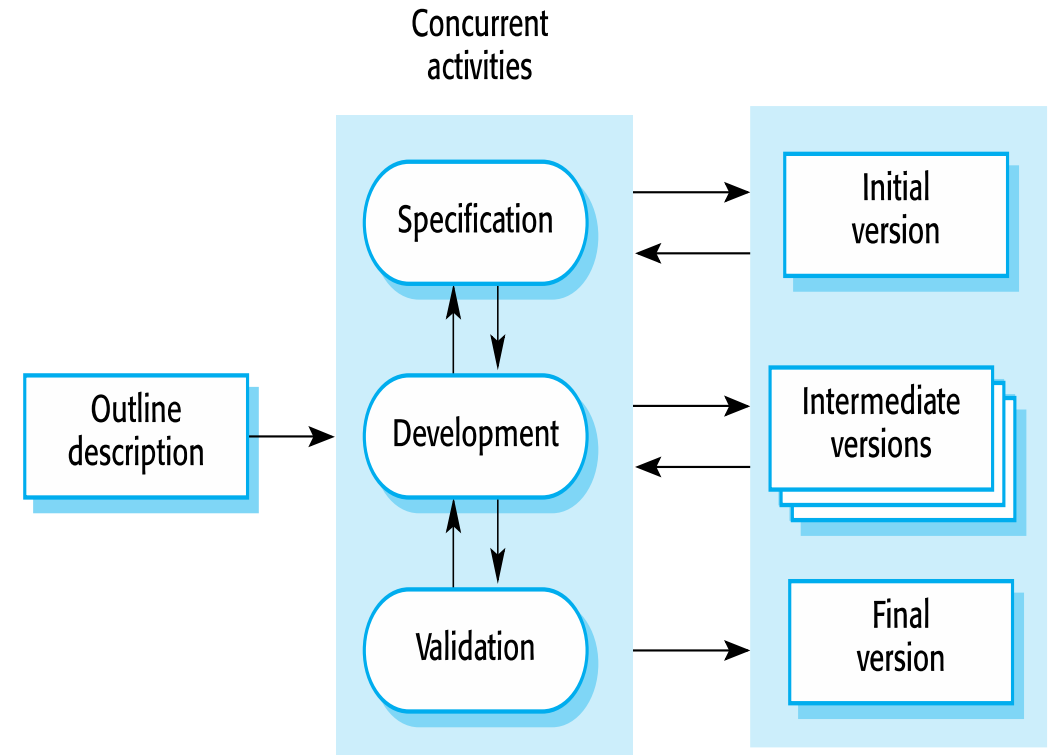
Incremental Development

- Incremental development is based on the idea of
 - developing an **initial implementation**,
 - exposing this to **user feedback**,
 - **evolving it through several versions** until an acceptable system has been developed.
- The **activities** of a process
 - **not separated**
 - **but interleaved** with feedback involved across those activities.



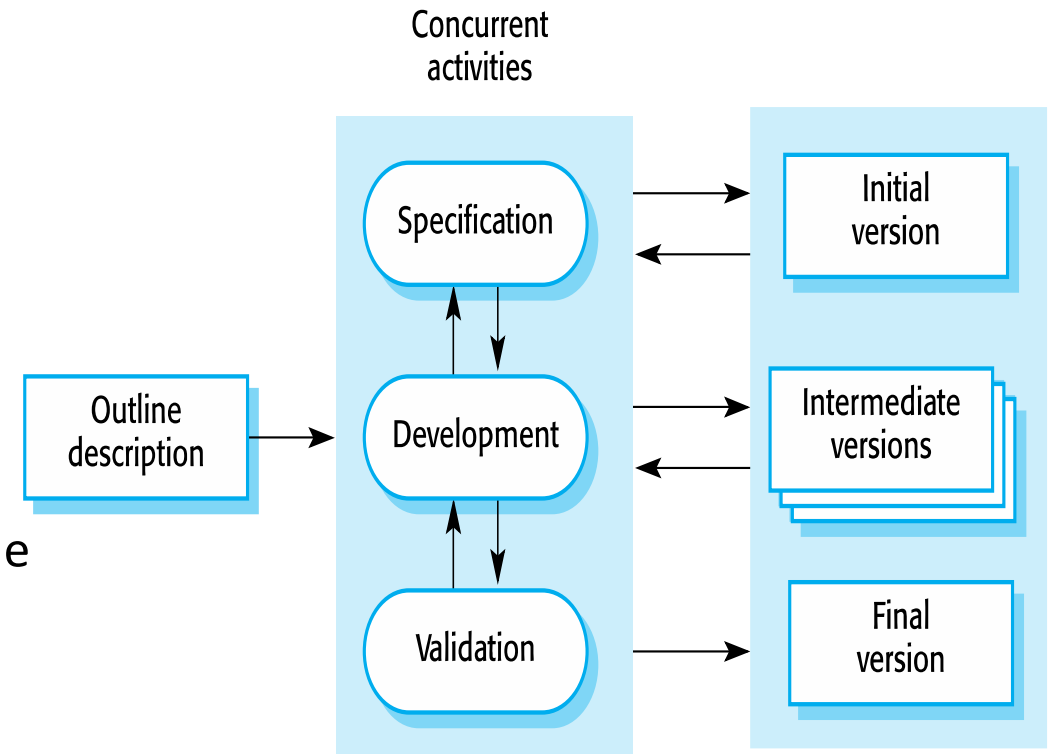
Incremental Development

- Each system increment reflects
 - a **needed piece of the functionality**
 - the early increments of the system
 - should include the **most important or most urgently** required **functionality**.
- **Customer can evaluate the system** at early stage in the development
 - see if it delivers what's required
 - If not,
 - only the current increment has to be changed,
 - and possibly, new functionality defined for later increments.



Incremental Development

- It can be a plan-driven or agile, or both
 - In a plan-driven approach,
 - the system increments are identified in advance
 - in the agile approach,
 - only the early increments are identified
 - the development of later increments depends on the progress and customer priorities.



Incremental Development: Benefits

1. The cost of implementing requirements changes is reduced.
2. Amount of documentation and analysis is reduced to a great extent.
3. It is easier to get customer feedback on the development work that has been done.
4. Early delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included.

Incremental Development: Problems

1. The process is not visible. Managers need regular deliverables to measure progress.

- If systems are developed quickly, it is not cost effective to produce documents that reflect every version of the system.

Incremental Development: Problems

2. System structure tends to degrade as new increments are added.
 - Regular change leads to messy code as new functionality is added.
 - It becomes increasingly difficult and costly to add new features to a system.
 - To reduce structural degradation,
 - Agile methods suggest that you should regularly refactor (improve and restructure) the software.

Incremental Development

- When to use?
 - When requirements are likely to change during the development process
 - This is the case for most business systems and software products.
 - When systems is not large, complex, long-lifetime systems, where different teams develop different parts of the system.

Reuse-oriented Software Engineering or Integration and Configuration

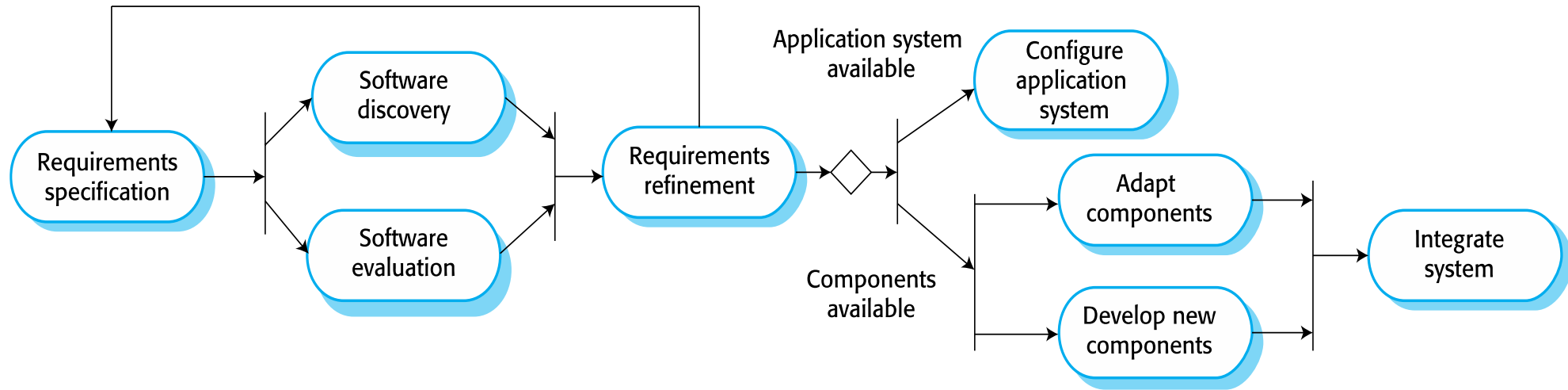
Reuse-Oriented Software Engineering

- Based on software reuse
 - where systems are integrated from existing **components** or **application** systems
(COTS -Commercial-off-the-shelf) systems).
- Reused elements may be **configured** to adapt their behaviour and functionality to a user's requirements
- Reuse is now the standard approach for building many types of business system

Reuse-Oriented Software Engineering

- Types of reusable software
 - Stand-alone application systems (sometimes called COTS) that are configured for use in a particular environment.
 - Collections of objects that are developed as a package to be integrated with a component framework such as .NET or J2EE.
 - Web services that are developed according to service standards and which are available for remote invocation.

Reuse-Oriented Software Engineering



1. Requirements specification -- initial requirements of the product are defined
2. Software discovery and evaluation – find software components and systems that provide required functionality
3. Requirements refinement – modify requirements on the basis of the reusable components available. If this is not possible go to step 2
4. Application system configuration – modify the application to provide the required functionality
5. Component adaptation and integration – if 4 is not possible, modify reusable components and integrate then

Reuse-Oriented Software Engineering

Benefits

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of system

Problems

- Loss of control over evolution of reused system elements – because the components being reused may not be under the control of the organization
- Requirements compromises are inevitable
 - so system may not meet real needs of users

Case Study

Case 1: Standardized HR & Payroll Modernization

- A regional enterprise wants to replace its end-of-life internal employee management system with modern software. The company operates standard human resource workflows such as timesheet tracking, standard bi-weekly payroll runs, compliance reporting, and benefits administration all of which adhere strictly to state labor regulations and off-the-shelf integration protocols. The IT budget and internal development capacity are strictly limited, making custom ground-up coding impractical. The technical team determines that standard commercial off-the-shelf (COTS) modules and established open-source identity-management libraries already satisfy 90% of their operational needs with minimal configuration.

Case Study

Case 2: Smart Campus Mobile Ecosystem

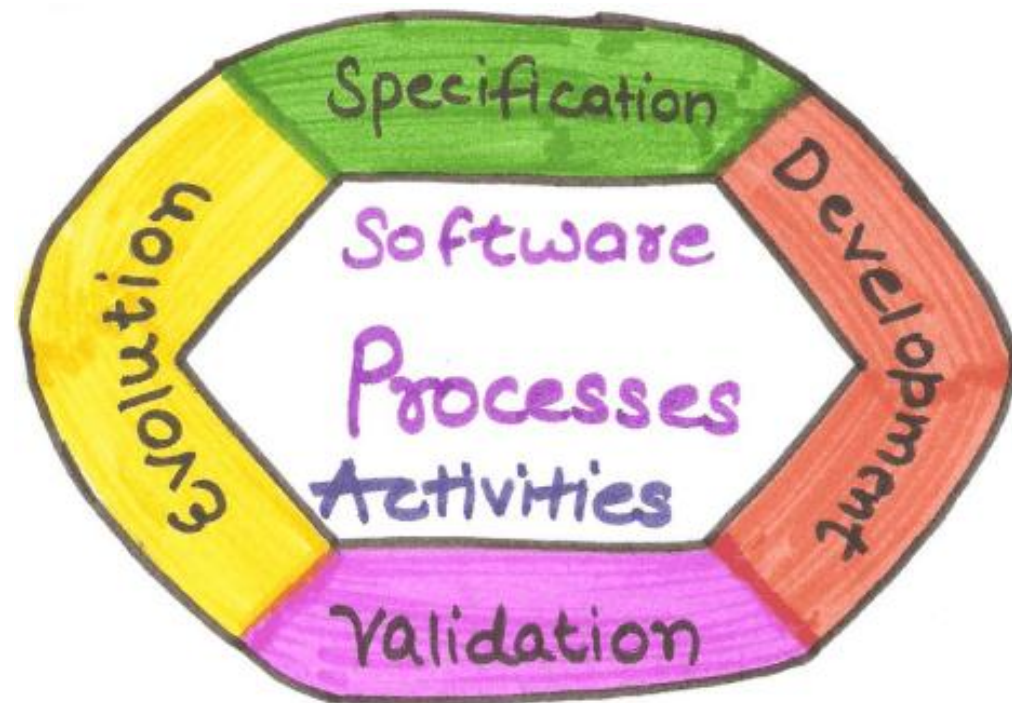
- A university plans to launch a unified digital hub for its campus community, encompassing digital student IDs, campus shuttle tracking, dining hall order-ahead services, and academic advising scheduling. While the full institutional scope is known, the leadership wants immediate utility on campus rather than waiting two years for a total release. The core digital ID and campus map must go live before the fall semester starts, after which the team will successively architect, validate, and roll out the dining service in the spring, followed by transportation and advising in subsequent release phases.

Case Study

Case 3: Railway Signaling & Track Interlocking System

- A transit authority is commissioning the automated track-switching and collision-prevention signaling software for a high-speed municipal rail corridor. The complete operational rules, track layout constraints, hardware control signals, and national rail safety standards are fully documented and signed off prior to implementation. Due to the catastrophic safety risks and fixed hardware interfaces, all requirements and system architecture must remain stable, with complete sign-off and formal verification required at the completion of each phase before engineering proceeds to the next.

Process activities



Process Activities

- Real software processes are inter-leaved sequences of
 - Technical, Collaborative, Managerial activities
- with the overall goal of
 - specifying,
 - designing,
 - implementing
 - and testing a software system.

Process Activities

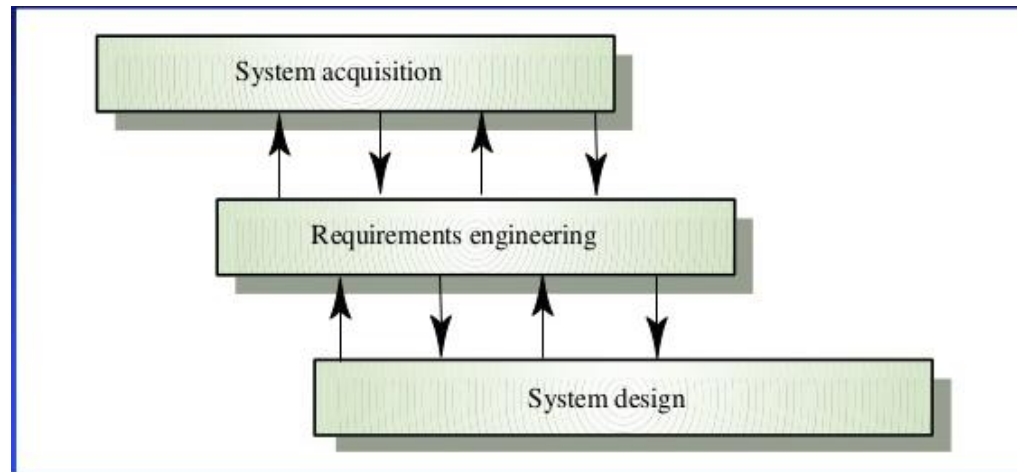
- The four basic process activities of
 - specification,
 - development,
 - validation
 - evolution
- They are organized differently in different development processes.
- For example
 - in the waterfall model, they are organized in sequence,
 - in incremental development they are interleaved.

The Requirements Engineering Process

- The process of establishing what
 - services are required
 - constraints on
 - the system's operation
 - development.
- Requirements engineering is a critical stage of the software process,
 - mistakes made at this stage inevitably lead to later problems in the system design and implementation.
- How to avoid mistakes?

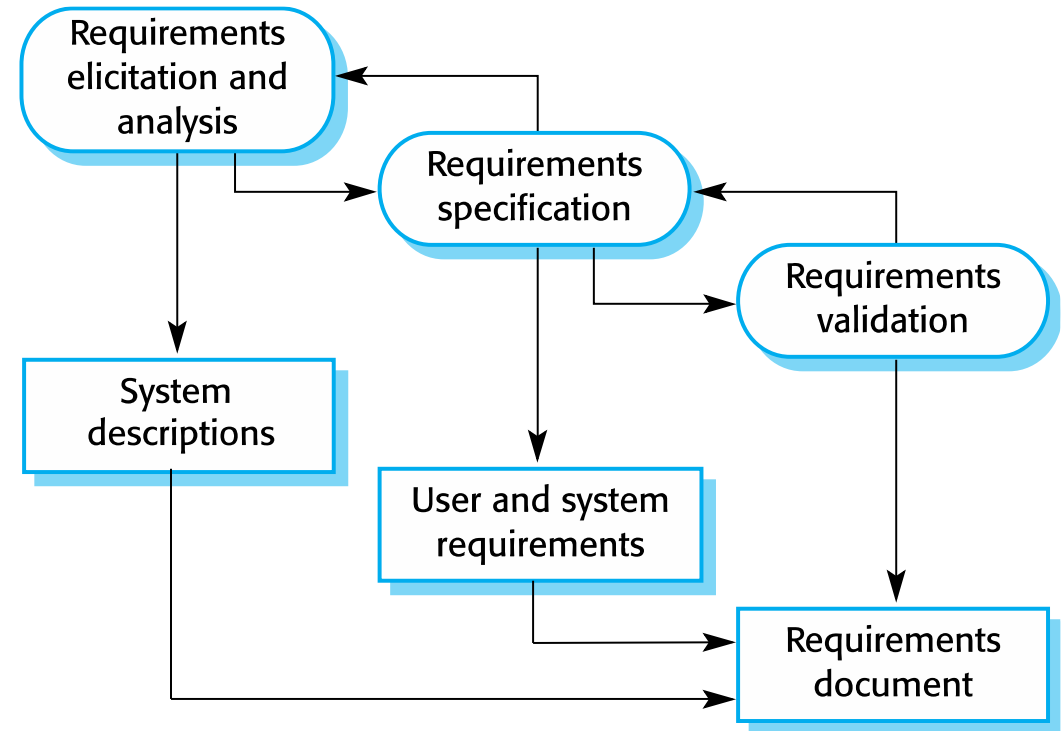
The Requirements Engineering Process

- **Feasibility** (marketing study):
 - to assess whether or not there is a need or a market for the software
 - whether or not it is technically and financially realistic to develop the software required.
- **Process Context :**



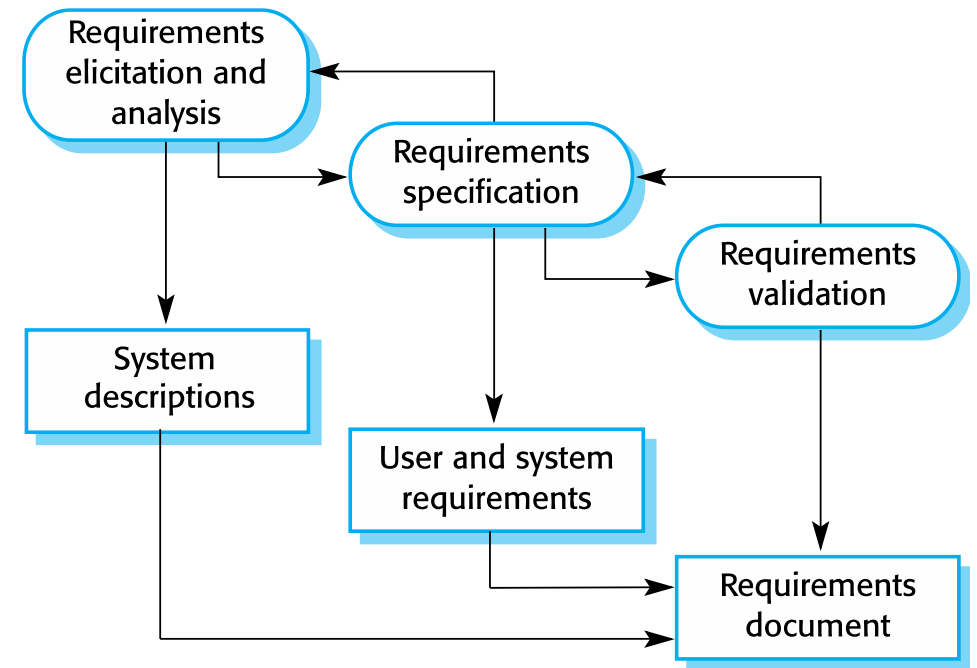
The Requirements Engineering Process

- Requirements engineering process?
 - Requirements elicitation and analysis
 - Requirements specification
 - Requirements validation



The Requirements Engineering Process

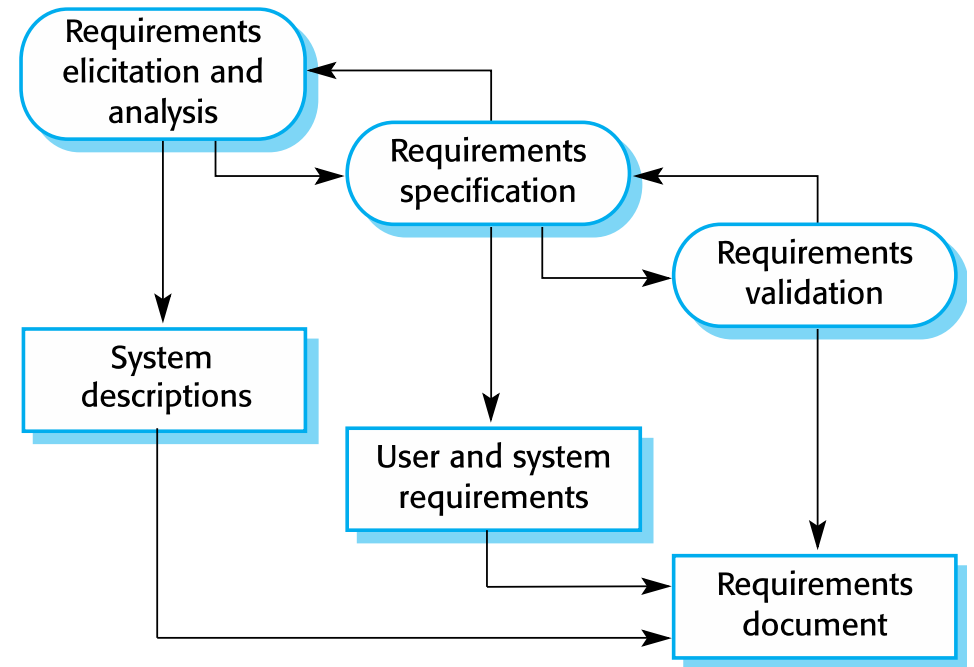
- Requirements elicitation and analysis
 - What do the system stakeholders require or expect from the system?
 - Deriving the system requirements through
 - observation of existing systems,
 - discussions with potential users and procurers,
 - task analysis, and so on.
- May develop one or more system models and prototypes.
 - These help you understand the system to be specified.



System developers need a more detailed system specification

The Requirements Engineering Process

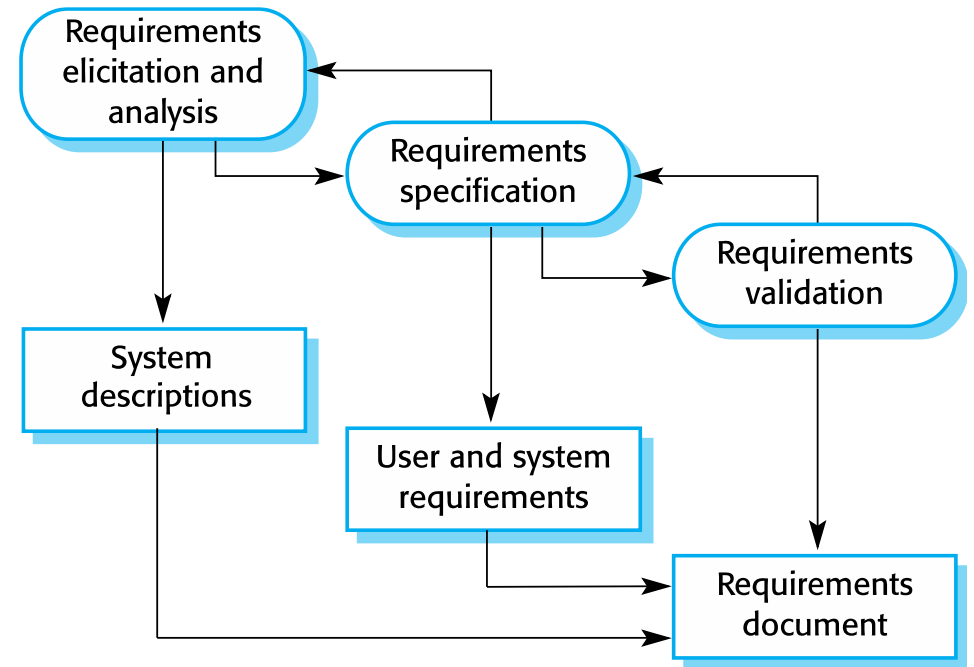
- Requirements specification
 - Requirements specification is the activity of translating the information gathered during requirements analysis into a set of requirements.
- Defining the requirements in detail
- Two types of requirements:
 - User requirements are abstract statements of the system requirements;
 - System requirements are a more detailed description of the functionality to be provided.



System developers need a more detailed system specification

The Requirements Engineering Process

- Requirements validation
 - Checking the validity of the requirements
- Errors in the requirements document are inevitably discovered.
 - It must then be modified to correct these problems.



System developers need a more detailed system specification

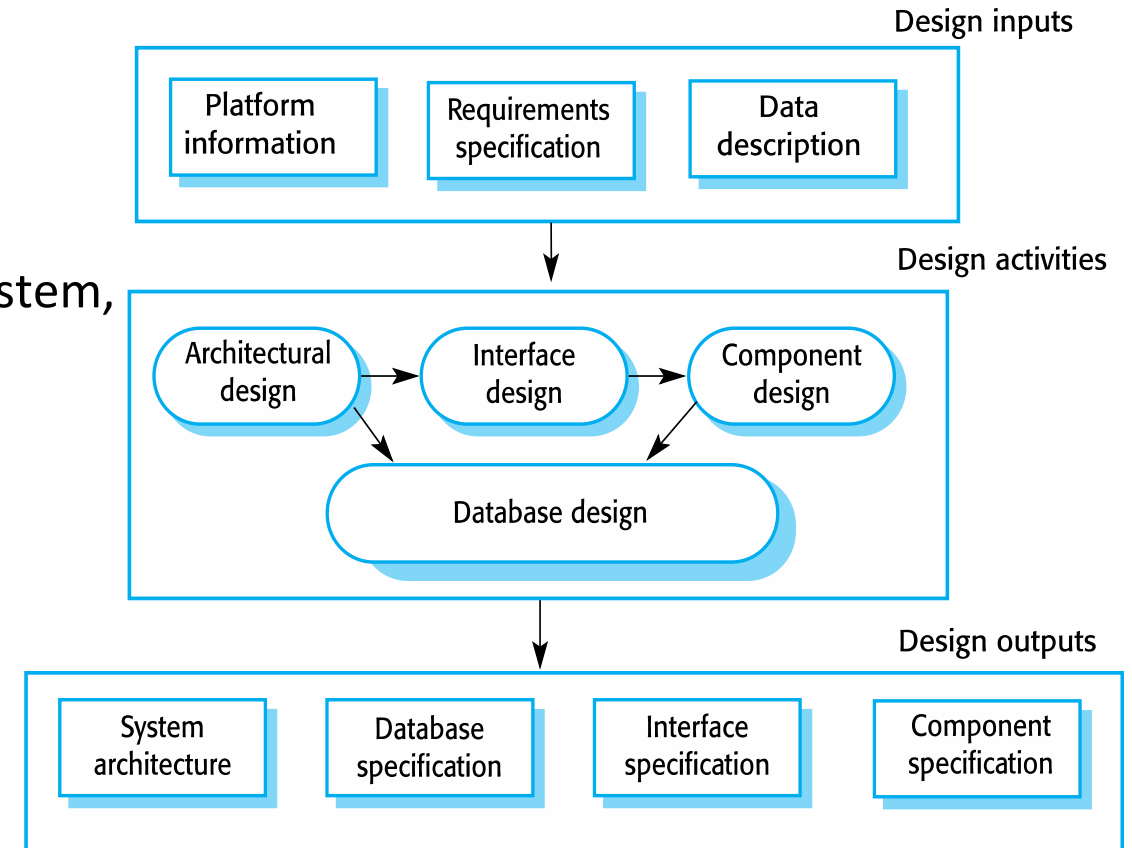
Software Design and Implementation

- The process of converting the system specification into an executable system.
- Software design
 - Design a software structure that realises the specification;
- Implementation
 - Translate this structure into an executable program;
- The activities of design and implementation are closely related and may be inter-leaved.

Design Activities

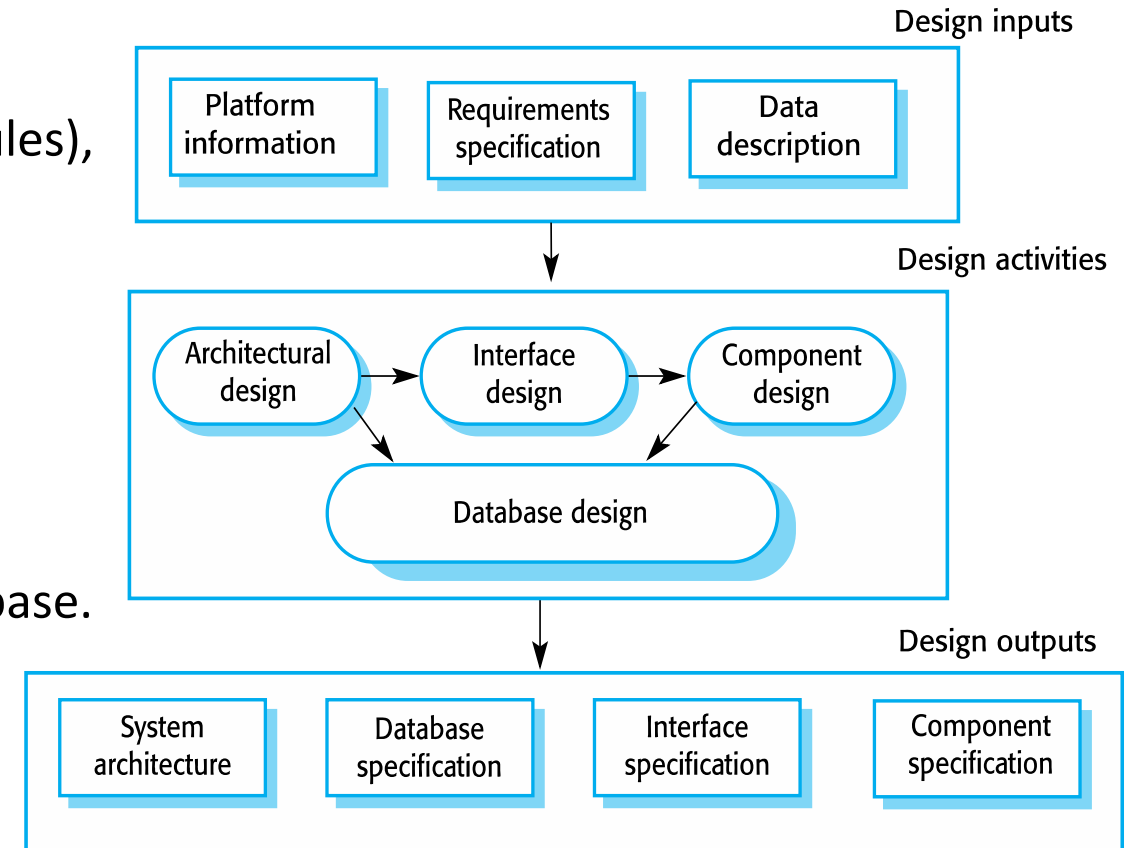
- **A general model of the design process**

- A software design is a description of
 - structure of the software to be implemented,
 - the data models and structures used by the system,
 - the interfaces between system components
 - sometimes, the algorithms used.



Design Activities

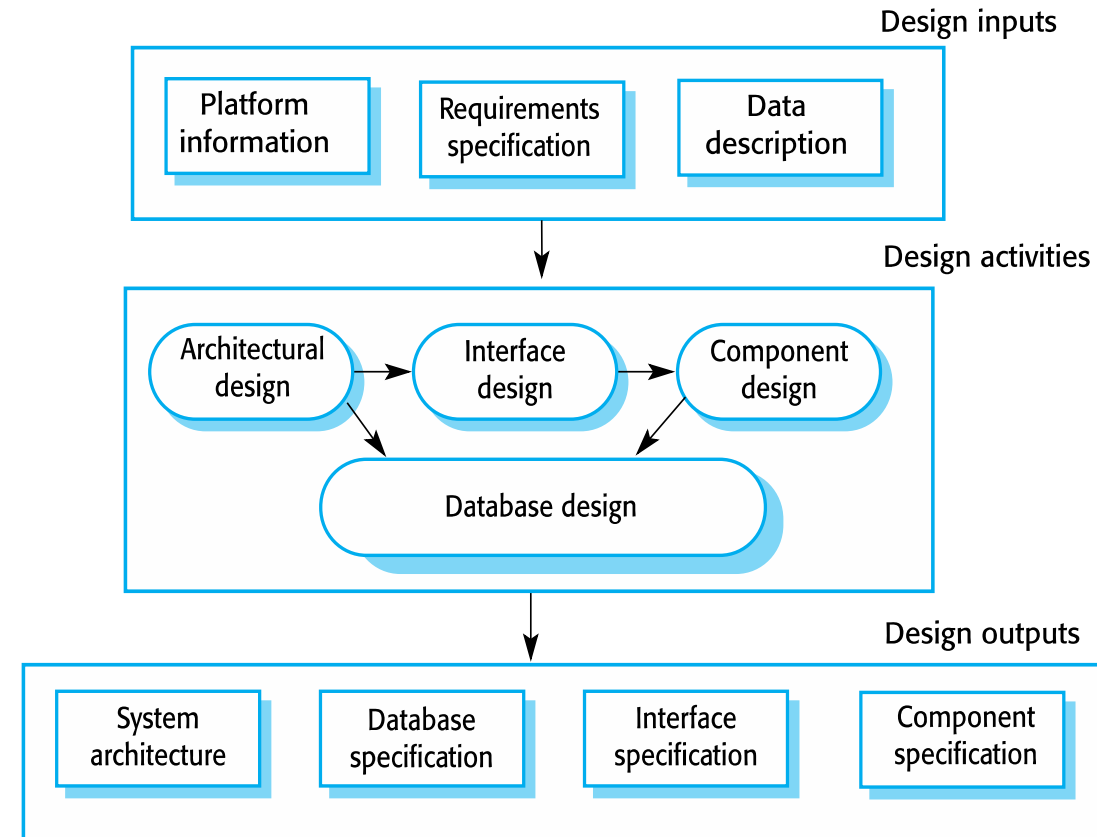
- Architectural design,
 - where you identify
 - the overall structure of the system,
 - the principal components (subsystems or modules),
 - their relationships
 - and how they are distributed.
- Database design,
 - where you design
 - the system data structures
 - and how these are to be represented in a database.



A general model of the design process

Design Activities

- Interface design,
 - where you define the interfaces between system components.
- Component selection and design,
 - where you search for reusable components.
 - If unavailable, you design how it will operate.



A general model of the design process

System implementation

- The software is implemented either by
 - developing a program or programs
 - configuring an application system.
- Programming is an individual activity with no standard process.
 - Start with components you understand and develop these
 - OR, developing the familiar components later
 - Some love to define data early, and use it to derive the program
 - others not, leave it unspecified as long as possible



System implementation

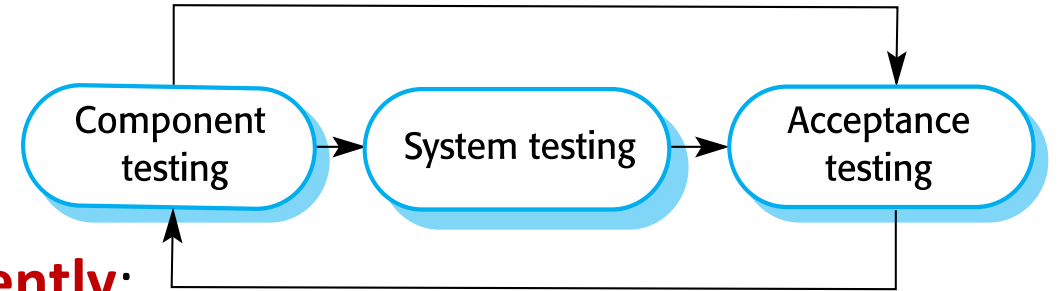
- Some developers carry some testing
- Testing reveals program defects
- Debugging is the activity of
 - finding program faults
 - correcting these faults.



Stages of Testing

- **Component testing**

- Individual **components are tested independently**;
- Components may be functions or objects or coherent groupings of these entities.



- **System testing**

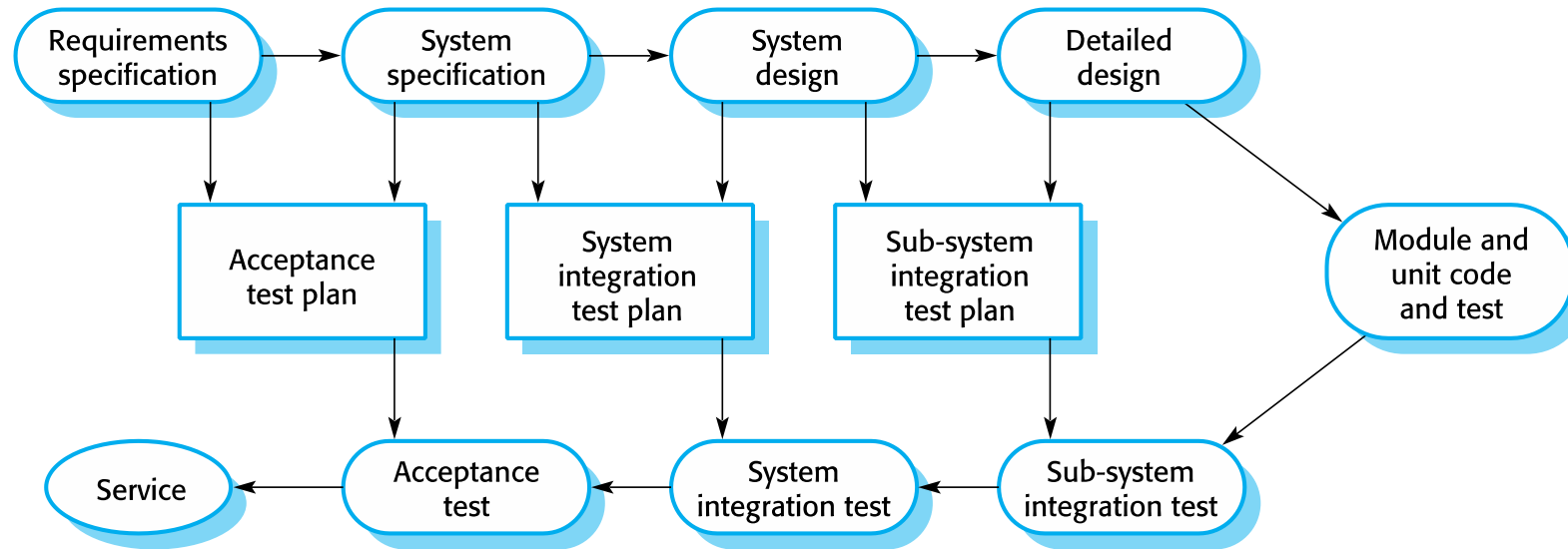
- Testing of the system as a whole.
- Testing of emergent properties is particularly important.

- **Customer / Acceptance testing**

- Testing with **customer data** to check that the system meets the customer's needs.

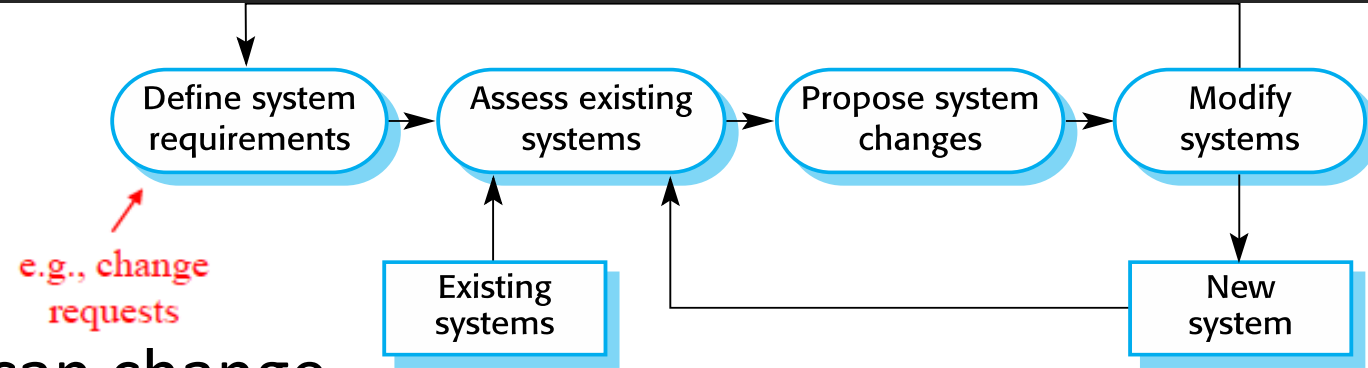
Stages of Testing

- Testing phases in a **plan-driven** software process (**V-model**)
 - Testing is driven by a set of test plans



Software Evolution (maintenance)

Software Evolution (maintenance)



- Software is inherently flexible and can change.
- As requirements change through changing business circumstances,
 - the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution
 - this is increasingly irrelevant as fewer and fewer systems are completely new.

Coping with change

Coping with Change

- Change is inevitable in all large software projects?
 - Business changes
 - lead to new and changed system requirements
 - New technologies
 - open up new possibilities for improving implementations
 - Changing platforms
 - require application changes
- Change leads to rework so the costs of change include both
 - Re-analyzing requirements
 - The costs of implementing new functionality

Reducing the Costs of Rework

Change anticipation

- includes process activities that can anticipate possible
 - user/stakeholder changes before development begins.
 - E.G., a prototype
 - may allow users/stakeholders to better envision how the system would actually be used.

SOLUTION

- System prototyping:
 - where a version of the system or part of the system is developed quickly to check
 - the customer's requirements and
 - the feasibility of design decisions.

Reducing the Costs of Rework

Change tolerance

- The process is designed so that changes can be accommodated at relatively low cost.
- This normally involves some form of incremental development.
 - Proposed changes may be implemented in increments that have not yet been developed.
 - If this is impossible, then
 - only a single increment (a small part of the system) may be altered to incorporate the change.

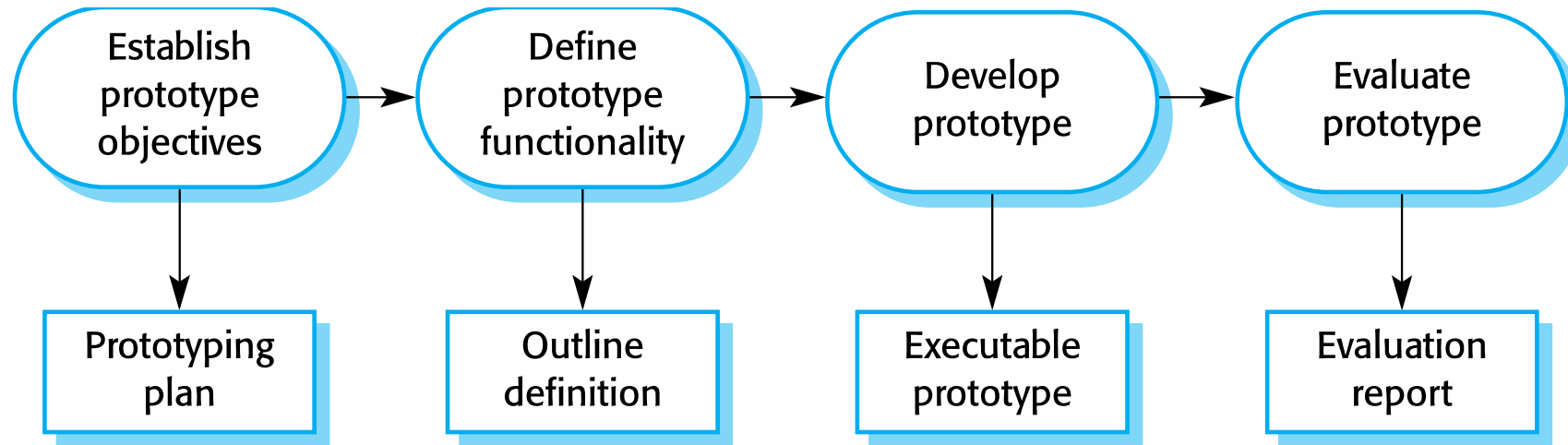
Coping with Changing Requirements

Software Prototyping

- A prototype is an initial version of a system used to
 - demonstrate concepts
 - try out design options.
- A prototype can be used in:
 - The requirements engineering process to help with requirements elicitation and validation;
 - In design processes to explore options and develop a UI design;
 - In the testing process to run back-to-back tests.

Software Prototyping

- Process of prototype development



Prototype Development

- May be based on rapid prototyping languages or tools
- May involve leaving out functionality
 - Prototype should focus on areas of the product that are not well-understood;
 - Error checking and recovery may not be included in the prototype;
 - Focus on functional rather than non-functional requirements such as reliability and security

Throw-away Prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system:
 - It may be impossible to tune the system to meet non-functional requirements;
 - Prototypes are normally undocumented;
 - The prototype structure is usually degraded through rapid change;
 - The prototype probably will not meet normal organizational quality standards.

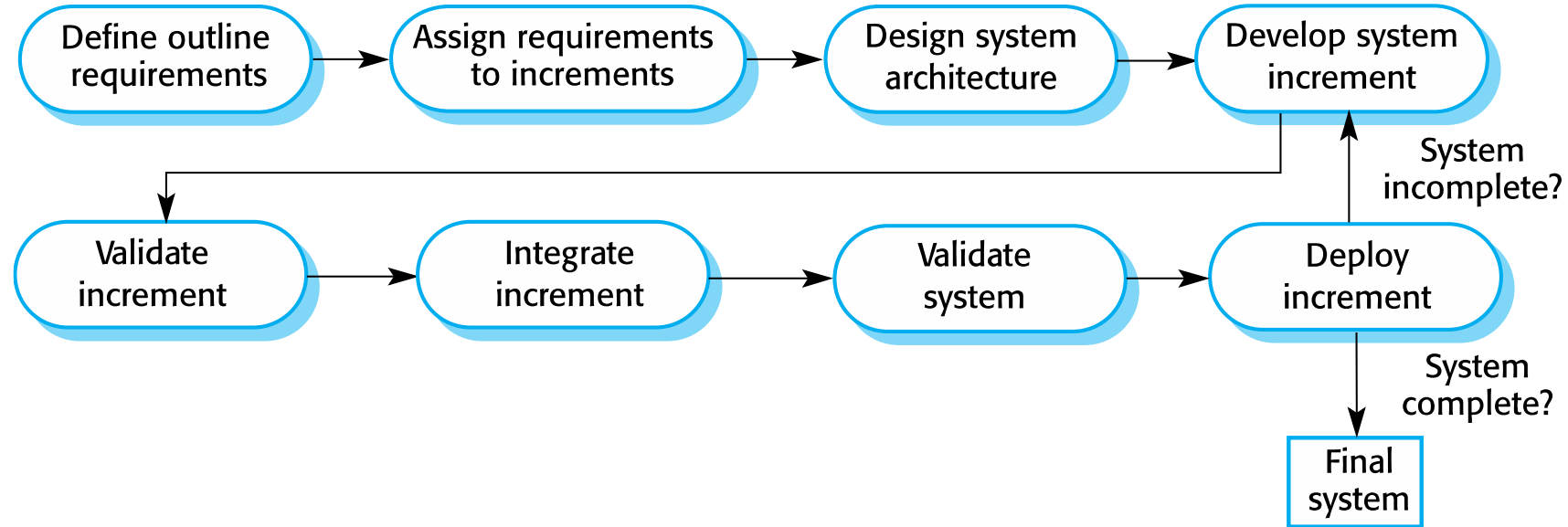
Benefits OF PROTOTYPING

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

Incremental Delivery

- Rather than deliver the system as a single delivery,
 - the development and delivery is broken down into increments
 - with each increment delivering part of the required functionality.
- User requirements are prioritised
 - the highest priority requirements are included in early increments.
- Once the development of an increment is started,
 - the requirements are frozen
 - though requirements for later increments can continue to evolve.

Incremental Delivery



Incremental Delivery Advantages

- Customer value can be delivered with each increment
 - so system functionality is available earlier.
- Early increments act as a prototype
 - to help elicit requirements for later increments.
- Lower risk of overall project failure.
- The highest priority system services tend to receive the most testing.

Incremental Delivery Disadvantages

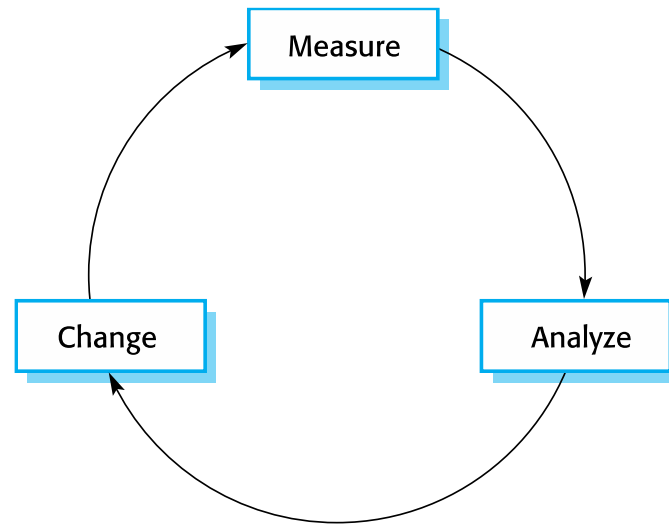
- Most systems require a set of basic facilities that are used by different parts of the system.
 - As requirements are not defined in detail until an increment is to be implemented,
 - it can be hard to identify common facilities that are needed by all increments.
- The essence of iterative processes is that the specification is developed in conjunction with the software.
 - However, this conflicts with the procurement model of many organizations,
 - where the complete system specification is part of the system development contract.

Process improvement

Process Improvement

- Many software companies have turned to software process improvement as a way of
 - enhancing the quality of their software,
 - reducing costs or accelerating their development processes.
- Process improvement means
 - understanding existing processes and
 - changing these processes
 - to increase product quality and/or reduce costs and development time.

The Process Improvement Cycle

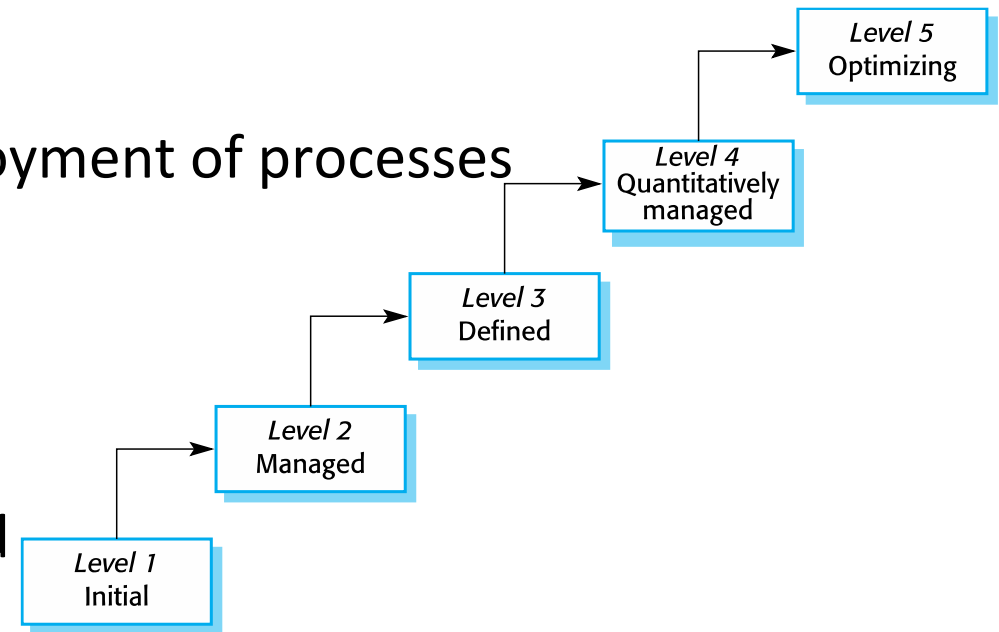


Approaches to Improvement

- The process maturity approach, focuses on
 - improving process and project management
 - introducing good software engineering practice.
 - The level of process maturity reflects the extent to which
 - good technical and management practice has been adopted in organizational software development processes.

The SEI capability maturity model

- Initial
 - For each process a scope of work is defined
- Managed
 - Product management procedures defined and used
- Defined
 - focus on organizational standardization and deployment of processes
- Quantitatively Managed
 - Quality management strategies defined and used
- Optimising
 - Process improvement strategies defined and used



Approaches to Improvement

- The agile approach, focuses on
 - iterative development and the reduction of overheads in the software process.
 - The primary characteristics of agile methods are
 - rapid delivery of functionality
 - responsiveness to changing customer requirements.

Key points

- Software processes are the activities involved in producing a software system.
- Software process models are abstract representations of these processes.
- General process models describe the organization of software processes.
 - Examples of these general models include the
 - 'waterfall' model,
 - incremental development, and
 - reuse-oriented development.

Key points

- Requirements engineering is the process of developing a software specification.
- Design and implementation processes are concerned with transforming a requirements specification into an executable software system.
- Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- Software evolution takes place when you change existing software systems to meet new requirements. The software must evolve to remain useful.

Key points

- Processes should include activities such as prototyping and incremental delivery to cope with change.
- Processes may be structured for iterative development and delivery so that changes may be made without disrupting the system as a whole.
- The principal approaches to process improvement are
 - agile approaches, geared to reducing process overheads, and
 - maturity-based approaches based on better process management and the use of good software engineering practice.
- The SEI process maturity framework identifies maturity levels that essentially correspond to the use of good software engineering practice.